Team 21: How Our Base Is Set Up

Oct 31, 2026 · @Forn

We have ACC-01 and ACC-06. The shared base is done and is the first 8 commits on both of our feature branches: our own projects, EF Core on Postgres, the result pattern, soft delete, a fake login for local work and a test project with 26 tests. Below is what's in it, how to run it and the few rules we all follow.

All commands are also in `docs/team-21/README.md` in the repo.

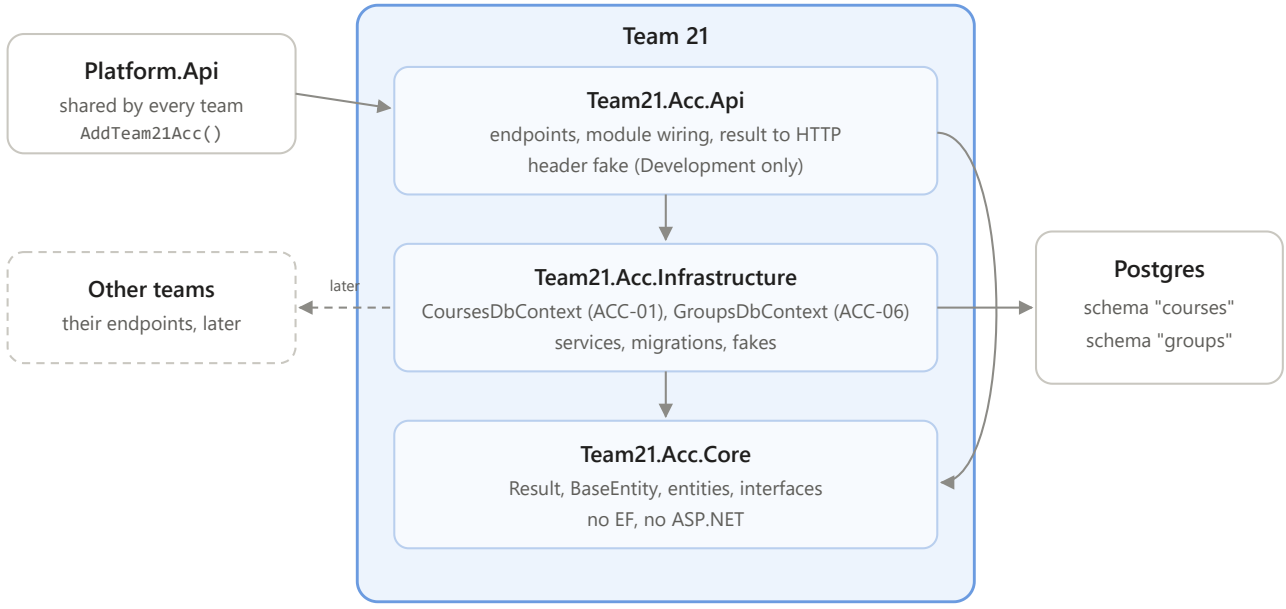
What we own

ID	Requirement	Acceptance criterion from the spec
ACC-01	Creating and archiving a course	Archiving stops new enrollments and does not delete history.
ACC-06	Groups and visibility per group	Members of one group do not see the closed materials of another.

Login, enrollment, materials, drafts, audit storage, data retention and CI are other teams' work. Where we need something from them we write a small interface for it and fake it until their endpoints are up.

How the pieces fit

Platform.Api calls two methods, the rest is in our three projects



Team 21 base · arrows point the way the code depends, never back up

Our code is split into three projects: Core, Infrastructure and Api. Platform.Api gets one project reference and two lines in `Program.cs`, our packages and code sit outside the files the other teams edit.

Project	Holds	Can see
`Team21.Acc.Core`	Result, Error, BaseEntity, entities, interfaces	nothing
`Team21.Acc.Infrastructure`	Both db contexts and their interfaces, services, migrations, fakes	Core, EF
`Team21.Acc.Api`	Endpoints, module wiring, result to HTTP, header fake	Core, Infrastructure, ASP.NET

HttpContext only exists in the Api project. The spec checks that a business module doesn't know a technical adapter (chapter 8.2), and with this split the compiler holds that for us.

ACC-01 and ACC-06 each have their own db context and Postgres schema, with separate migrations. Both sub teams can add tables at the same time.

Who does what

#	Part	Task	Branch
1	ACC-01	Course model, database, migrations, test data	`feature/acc01-model`
2	ACC-01	Backend API for creating and archiving	`feature/acc01-api`
3	ACC-01	Frontend for courses	`feature/acc01-frontend`
4	ACC-06	Group model, memberships, link to materials	`feature/acc06-model`
5	ACC-06	Backend for managing groups	`feature/acc06-groups-api`
6	ACC-06	Server side visibility of materials	`feature/acc06-visibility`
7	ACC-06	Frontend for groups and materials	`feature/acc06-frontend`
8	Both	Automated, negative and integration tests	`feature/acc01-tests`, `feature/acc06-tests`
9	Both	Integration with other teams, contracts, audit	`feature/acc-integration`, later

Everyone integrates and tests their own code. Member 9 looks after the shared contracts and swaps the fakes for real adapters when other teams have endpoints.

Branches

```
main
 ├─ feature/acc01-courses           ACC-01
 │   ├─ feature/acc01-model
 │   ├─ feature/acc01-api
 │   ├─ feature/acc01-frontend
 │   └─ feature/acc01-tests
 └─ feature/acc06-groups            ACC-06
     ├─ feature/acc06-model
     ├─ feature/acc06-groups-api
     ├─ feature/acc06-visibility
     ├─ feature/acc06-frontend
     └─ feature/acc06-tests
```

Both feature branches start with the same 8 base commits. Integration with other teams gets `feature/acc-integration` off `main` once both features have something to connect.

- **Your branch** comes off the feature you work on: `git switch feature/acc06-groups`, then `git switch -c feature/acc06-visibility`.
- **Handing in** is a pull request into the feature branch. The team lead reviews it.
- **Picking up new work** from the feature branch: `git merge feature/acc06-groups`.
- **Merge, not rebase.** Rebase rewrites history and can drop a teammate's commits.

Sub branches use a hyphen, `feature/acc06-visibility`. Git can't have `feature/acc06-groups` and `feature/acc06-groups/visibility` at the same time.

Getting it running

You need the .NET 10 SDK, Node 22.12 or newer, and Docker.

1. Start Postgres. Copy `.env.example` to `.env` in the repo root and set a password.

```
docker compose up -d db
```

2. Put the connection string in user secrets, with the values from your `.env`.

```
dotnet user-secrets set "ConnectionStrings:Team21"
"Host=localhost;Port=5432;Database=platform;Username=platform;Password=YOUR_PASSWORD" --
project backend/Team21.Acc.Infrastructure
```

3. Install the EF tool.

```
dotnet tool install --global dotnet-ef
```

4. Build, test and run. `backend/Team21.Acc.slnx` also opens in Visual Studio or Rider.

```
dotnet build backend/Team21.Acc.slnx
dotnet test backend/Team21.Acc.Tests
dotnet run --project backend/Platform.Api --urls http://localhost:5080 -- --environment
Development
```

Platform.Api has no launch settings, so without `--environment Development` it starts in Production. Production skips our user secrets and the header fake. Without the connection string the api still starts, only our endpoints fail.

5. Check it's up.

```
curl http://localhost:5080/api/health
```

6. Call an endpoint as someone. No real login yet, so in Development you send the user id in a header. Once ACC-01 has endpoints a call looks like this. Outside Development the header is ignored.

```
curl -H "X-Test-User: 10000000-0000-0000-0000-000000000001"
http://localhost:5080/api/courses
```

What the base gives you

Piece	What it is	In practice
`Result<T>`, `Error`	What a service hands back, the value or an error (Core)	`return Error.NotFound("Course not found");` or just `return dto;`
`ToHttpResult()`	Result to HTTP response (Api)	200 or 204 on success, ProblemDetails with the matching status on failure. `ToCreatedHttpResult()` gives 201
`BaseEntity`	Base class for our tables (Core)	`Id`, created, updated and deleted fields, `IsDeleted`
`TeamDbContextBase`	What both db contexts share (Infrastructure)	Own schema, UTC dates, `CreatedAt` and `UpdatedAt` on save, the soft delete filter, configurations picked up on their own
`ICoursesDbContext`, `IGroupsDbContext`	What services inject (Infrastructure)	The concrete context is never injected
`ICurrentActor`	Who is making the request (Core)	`actor.UserId`, null when nobody is signed in
`CoursesModule`, `GroupsModule`	Where each feature registers services and endpoints (Api)	The files you touch to plug your work in

Piece	What it is	In practice
`Team21.Acc.Tests`	NUnit project with 26 tests	Your tests go next to them, in the folder of the project you test

Writing a service and an endpoint

The service interface goes in Core, the implementation in Infrastructure. It injects the context interface and the current actor, checks the caller first and returns a result.

```
public CourseService(ICoursesDbContext context, ICurrentActor actor)

if (!_actor.IsAuthenticated)
    return Error.Unauthorized("Sign in first");
```

Endpoints are minimal APIs in the Api project, same as the course template and the other teams. An endpoint is a route plus a function. ASP.NET fills the function's parameters from the route (`{id}`), the query string, the JSON body and dependency injection.

```
public static class CourseEndpoints
{
    public static IEndpointRouteBuilder MapCourseEndpoints(this IEndpointRouteBuilder
app)
    {
        var group = app.MapGroup("/api/courses");
        group.MapGet("/{id:guid}", GetCourse);
        return app;
    }

    private static async Task<IResult> GetCourse(Guid id, ICourseService courses,
CancellationToken ct)
        => (await courses.GetAsync(id, ct)).ToHttpResult();
}
```

Keep endpoints in a static class like this and call it from your module file. The endpoint reads the input, calls the service and returns `ToHttpResult()`. Access is checked in the service on every request, hiding a button in the frontend doesn't count.

Adding a table

1. Write the entity in Core under your feature, for example `Team21.Acc.Core/Courses/Course.cs`, deriving from `BaseEntity`.

2. Add a `DbSet` to your feature's context and its interface, `CoursesDbContext` and `ICoursesDbContext`.

3. Keys, indexes and lengths go in an `IEntityTypeConfiguration<T>` class in Infrastructure under your feature. It gets picked up on its own.

4. Create the migration with the context and folder named.

```
dotnet ef migrations add AddGroups --context GroupsDbContext --output-dir
Groups/Data/Migrations --project backend/Team21.Acc.Infrastructure --startup-project
backend/Team21.Acc.Infrastructure
dotnet ef database update --context GroupsDbContext --project
backend/Team21.Acc.Infrastructure --startup-project backend/Team21.Acc.Infrastructure
```

5. Read the generated file before you commit it. A renamed property can come out as drop column plus add column. A migration someone already applied stays as it is, changes go in a new one.

What every table gets

- **CreatedAt and UpdatedAt** are filled on save, sync and async.
- **CreatedBy and UpdatedBy** are set in the service from the current caller: `CreatedBy = userId`.
- **CreatedAt and CreatedBy** can't be changed by an update, a save puts them back.
- **Soft delete** is done in the service. Rows with `IsDeleted` drop out of every query, and stay readable with `.IgnoreQueryFilters([TeamDbContextBase.SoftDeleteFilter])`.
- **UTC dates.** Every `DateTime` is stored as UTC. The frontend sends dates with `toISOString()`.

```
member.IsDeleted = true;
member.DeletedAt = DateTime.UtcNow;
member.DeletedBy = userId;
await _context.SaveChangesAsync();
```

Things to keep in mind:

- `Remove()` really deletes the row. On our tables we soft delete instead.
- A unique index needs `.HasFilter("\"IsDeleted\" = false")`, otherwise a removed row blocks adding it back.
- An archived course is a normal row with an archived status. ACC-01 wants the history kept.
- The two schemas have no foreign keys between them. ACC-06 keeps `CourseId` as a plain column and asks ACC-01 about the course through an interface.

What is faked

Only the `X-Test-User` header exists so far. The rest are planned: whoever first needs one writes the interface and its fake next to their code, registered in Development only.

Fake	Stands in for	Owner	Status
`X-Test-User` header	Login and sessions	IDN teams	done
Account status	Active or blocked accounts	IDN-03 team	planned
Course membership and role	Enrollment, teacher assignment	ACC-02 and ACC-03 teams	planned
Material catalog	Labs and assignments	LAB and ASN teams	planned
Audit writer	Audit log storage	AUD team	planned

Outside Development nobody is signed in. Every service checks `IsAuthenticated`, so every request there gets 401. A fake is a test double and never counts as a finished integration, member 9 replaces them once the other teams have endpoints.

Common mistakes

Mistake	Symptom	Fix
`dotnet ef` without `--context`	"More than one DbContext was found"	Name the context and the output folder
Packages added to `Platform.Api.csproj`	Merge conflicts with other teams	Add them to the Team21 project that needs them
HttpContext or EF used in Core	Core stops building	HTTP stays in Api, EF in Infrastructure
Injecting `CoursesDbContext` in a service	Service tied to the concrete context	Inject `ICoursesDbContext`
Service without the `IsAuthenticated` check	Anonymous callers get through	Return `Error.Unauthorized` first
Connection string in `appsettings.json`	Password ends up in git history	User secrets, step 2 above
Rebasing a shared branch	Teammates' commits go missing or show up twice	Merge
Unique index without the `IsDeleted` filter	A removed member can't be added back	`.HasFilter("\"IsDeleted\" = false")`
`Remove()` or `ExecuteDeleteAsync` on our tables	Rows really deleted, history gone	Set `IsDeleted`, `DeletedAt` and `DeletedBy`
`ExecuteUpdateAsync` without stamps	No updated time on those rows	Set `UpdatedAt` in the setters

Mistake	Symptom	Fix
Forgetting `CreatedBy` on insert	Rows with an empty creator	`CreatedBy = userId` in the service
Client dates without a zone	Npgsql throws on save, or times shift	`toISOString()` on the frontend
Throwing for "not found" or "not allowed"	500 errors and stack traces	Return an `Error` from the service

Commits

Short, area in brackets, one change per commit.

```
feat(acc06): group members load once per request
fix(acc01): archived course rejects new enrollments
test(acc06): a member of group A cannot read group B material
docs(team21): migration commands in the readme
```

Decisions so far

- Our code lives in `Team21.Acc.Core`, `.Infrastructure` and `.Api`.
- EF Core 10 on Postgres, two db contexts, two schemas, services inject the context interfaces.
- Minimal APIs and the result pattern for every endpoint.
- Soft delete on every table, done in the service. Archiving a course is a status.
- Other teams are faked behind our own interfaces, member 9 swaps them later.
- In ACC-06 a material a student can't see answers 404, same as one that doesn't exist.
- NUnit for tests, as the spec asks.

Next steps

1. Everyone pulls `feature/acc01-courses` or `feature/acc06-groups` and runs the setup once.
2. Members 1 and 4 add the first tables and migrations for courses and groups.
3. Members 4, 5 and 6 agree on the interface ACC-06 reads groups through. 5 and 6 work on fakes until 4 is done.
4. Everyone makes their own branch and fills in their KT1 component document in `docs/team-21/`.